If you specify multiple potential matches for a value, the last one defined wins. If I define this:

```
td.when(fake.remoteInit(anything())).thenReturn(1)
td.when(fake.remoteInit(isA(Number))).thenReturn(2)
td.when(fake.remoteInit(1)).thenReturn(3)
```

then fake.remoteInit(1) returns 3, fake.remoteInit(2) returns 2 (because 2 is a number but it isn't 1), and fake.remoteInit("Steve") returns 1 (because the only definition that matches is the first one).

You can also trigger different behaviors, typically to allow for different kinds of JavaScript asynchronous behavior. So if the method under test is expected to return a JavaScript promise, you can chain the test double with thenResolve to trigger the positive, with then to branch off the promise, or with thenReject to trigger the negative or catch side.

More generic asynchronous behavior can be captured with thenCallback, which assumes by default that the last argument to the function under test is itself a callback function, and which then invokes the callback function with whatever arguments you pass to thenCallback. You can also simulate an error condition with thenThrow, which expects an Error object.

Here's the when line from your original test:

```
td.when(FakeLoader.prototype.load()).thenResolve(input)
```

Now you can trace all of this. Here you're saying that FakeLoader is a class with an instance method load. If load is called with no arguments, you'd expect it to return a promise and immediately invoke the positive result of that promise.

To see what that does in practice, let's look at the passing code in Project and ProjectLoader:

js_jasmine/02/app/javascript/packs/project.js

```
import {ProjectLoader} from "../../../app/javascript/packs/project_loader.js"
import {Task} from "../../../app/javascript/packs/task.js"
import {ProjectTable} from "../../../app/javascript/packs/project_table.js"

export class Project {
  constructor(id) {
    this.tasks = []
    this.id = id
    this.loader = new ProjectLoader(this)
  }

  load() {
    return this.loader.load().then(data => this.loadFromData(data))
  }
```